

# SMART HAS

## Inteligência de Dados com Oracle PL/SQL

*AI Logistics Extension · React Native · Spring Boot · Angular · Oracle PL/SQL*

### Documentação da Atividade

FIAP — Fase 6 - Mastering Relational and Non-Relational Database — 06/10/2026

#### Integrante

| Nome completo        | RM     | Turma |
|----------------------|--------|-------|
| Leonardo Marsiarelli | 555366 | 3SIOA |

*Entrega individual.*

# 1. Contextualização

---

Nas fases anteriores, o Smart HAS evoluiu de um escopo planejado para um MVP funcional e conectado: app mobile em React Native, API REST em Spring Boot com autenticação JWT e a camada AI Logistics Extension (motor de regras que prioriza entrega de medicamentos), e um dashboard administrativo em Angular — todos integrados pelo mesmo contrato de API.

Nesta fase, o foco é elevar a robustez e o profissionalismo do sistema adotando o Oracle PL/SQL: construímos procedures e functions reutilizáveis diretamente no banco de dados e integramos essa camada lógica ao back-end Java, consolidando a interoperabilidade entre mobile, web, API REST e banco Oracle.

## 2. Parte 1 — Aprimoramento da Solução

---

A evolução técnica desta fase é a própria camada de inteligência Oracle descrita nas Partes 2 e 3: ela foi integrada ao back-end existente sem quebrar o que já funcionava, seguindo boas práticas arquiteturais.

### Novas funcionalidades

- Quatro novos endpoints REST (`/api/oracle/**`) que expõem as functions e procedures PL/SQL ao restante do sistema.
- Réplica automática de leituras críticas para o Oracle a cada `POST /api/readings`, com registro de alerta via `PRC_REGISTER_CRITICAL_ALERT` — fluxo REST → Java → JDBC → Oracle funcionando de ponta a ponta.
- Relatório de consumo/entregas por paciente (`PRC_USER_DELIVERY_REPORT`), disponível via API para consumo futuro pelo dashboard ou pelo app.

### Refatoração e boas práticas

- Datasource Oracle isolado (`OracleDataSourceConfig`), com injeção opcional (`Optional<OracleIntelligenceService>`) em `BloodPressureService` — a camada Oracle é um complemento plugável, nunca uma dependência obrigatória do fluxo principal.
- Falha na réplica Oracle é tratada como best-effort (log de WARN) e nunca derruba a criação da leitura no datasource operacional (H2/Postgres).
- Mesma regra de classificação de pressão (SBC) implementada nos dois lados — Java (`LogisticsAiService`) e PL/SQL (`FN_CLASSIFY_BP`) — com os mesmos limiares.

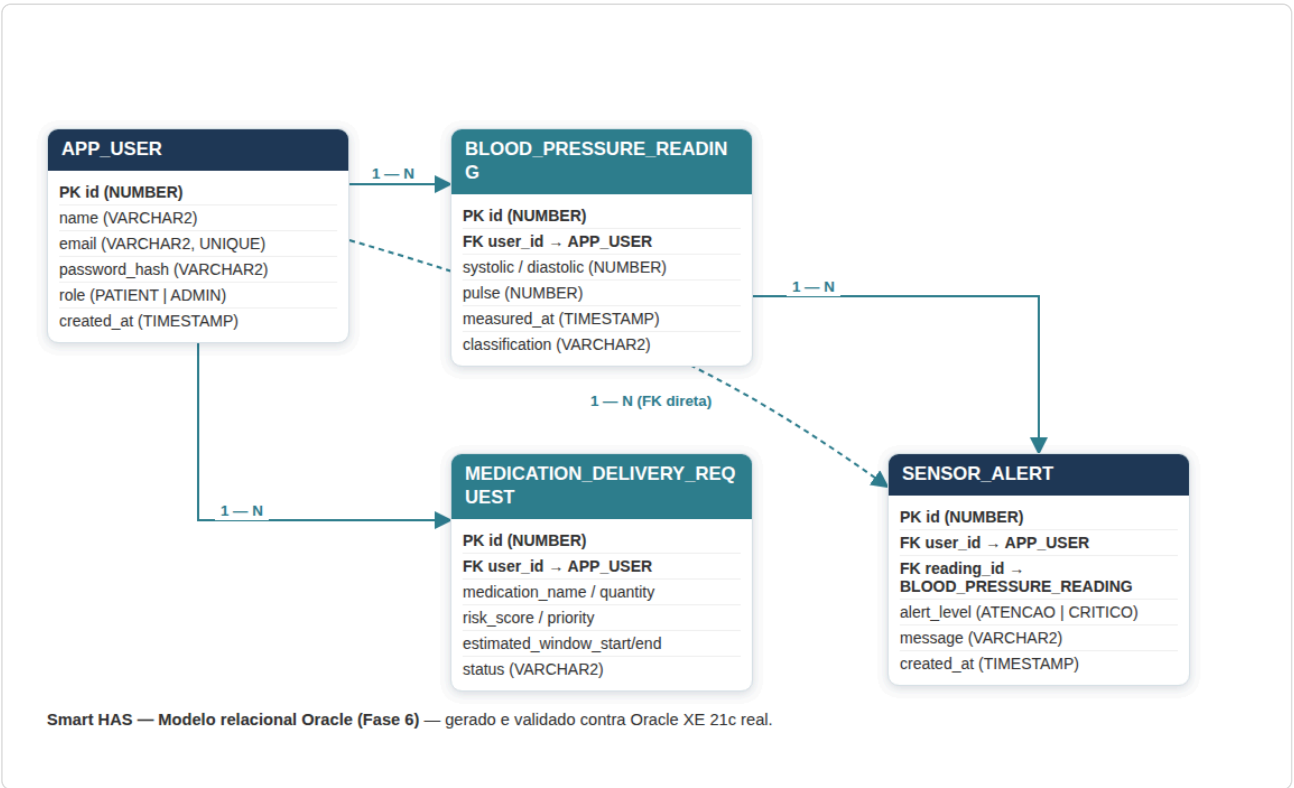
### Valor agregado

O sistema ganha uma camada de regras de negócio reutilizável diretamente no banco — qualquer cliente que converse com o Oracle (relatórios, BI, outro serviço) pode reaproveitar as mesmas regras de classificação, risco e alerta sem reimplementá-las em outra linguagem.

## 3. Parte 2 — Integração do Banco Oracle com o Smart HAS

### 3.1 Modelo lógico/físico

Quatro tabelas representam o domínio: **APP\_USER**, **BLOOD\_PRESSURE\_READING** (leituras de sensor), **MEDICATION\_DELIVERY\_REQUEST** (AI Logistics Extension) e **SENSOR\_ALERT** (nova nesta fase, alimentada pelas procedures).



DER — Modelo relacional Oracle (Fase 6).

### 3.2 Tabelas implantadas

| Tabela                      | Descrição                                                                           |
|-----------------------------|-------------------------------------------------------------------------------------|
| APP_USER                    | Pacientes e administradores (id, name, email, password_hash, role, created_at)      |
| BLOOD_PRESSURE_READING      | Leituras de sensor de pressão arterial (systolic, diastolic, pulse, classification) |
| MEDICATION_DELIVERY_REQUEST | Pedidos de entrega da AI Logistics Extension (risk_score, priority, status)         |
| SENSOR_ALERT                | Alertas gerados por PRC_REGISTER_CRITICAL_ALERT a partir de leituras críticas       |

### 3.3 Dados simulados

O script 02\_seed\_data.sql popula 3 usuários (1 admin + 2 pacientes), um histórico de 6 leituras de sensor ao longo do tempo — cobrindo todas as classificações, da NORMAL até CRISE\_HIPERTENSIVA — e 3 pedidos de entrega de medicamento, usados nos exemplos práticos da Parte 3.

## 4. Parte 3 — Functions e Procedures PL/SQL

### 4.1 Functions

| Function            | Parâmetros                       | Retorno            | Propósito                                                    |
|---------------------|----------------------------------|--------------------|--------------------------------------------------------------|
| FN_CLASSIFY_BP      | p_systolic IN,<br>p_diastolic IN | VARCHAR2           | Classifica a pressão arterial (reutilizada pelas demais)     |
| FN_CALC_RISK_SCORE  | p_systolic IN,<br>p_diastolic IN | NUMBER             | Indicador de risco (0-100) usado pela AI Logistics Extension |
| FN_GET_USER_SUMMARY | p_user_id IN                     | VARCHAR2<br>CURSOR | Dados formatados: resumo do paciente via CURSOR              |

Exemplo de uso em uma consulta SQL:

```
SELECT u.name, fn_classify_bp(r.systolic, r.diastolic) AS classificacao,  
       fn_calc_risk_score(r.systolic, r.diastolic) AS risk_score  
FROM blood_pressure_reading r JOIN app_user u ON u.id = r.user_id;
```

Todas as três tratam exceções (RAISE\_APPLICATION\_ERROR com códigos -20010 a -20013), têm parâmetros IN e RETURN explícitos, e comentários descrevendo propósito/parâmetros/retorno (ver database/03\_functions.sql).

### 4.2 Procedures

| Procedure                   | Parâmetros                                                                                | Conceitos                     | Propósito                                                                                                                                                          |
|-----------------------------|-------------------------------------------------------------------------------------------|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PRC_REGISTER_CRITICAL_ALERT | p_reading_id IN                                                                           | IF,<br>EXCEPTION              | Registra alerta em SENSOR_ALERT quando a leitura é HAS_ESTAGIO_2/CRISE_HIPERTENSIVA. Acionada pelo backend Java (REST → Java → JDBC → Oracle) a cada nova leitura. |
| PRC_USER_DELIVERY_REPORT    | p_user_id IN,<br>p_total_count OUT,<br>p_total_qty OUT,<br>p_details OUT<br>SYS_REFCURSOR | CURSOR,<br>LOOP,<br>EXCEPTION | Relatório resumido de entregas por paciente: total de pedidos/unidades via cursor explícito + detalhe via ref cursor.                                              |

Exemplo de execução (PL/SQL):

```
DECLARE v_total NUMBER; v_qty NUMBER; v_cur SYS_REFCURSOR;  
BEGIN  
  prc_user_delivery_report(3, v_total, v_qty, v_cur);  
  DBMS_OUTPUT.PUT_LINE('Total: ' || v_total || ' | Unidades: ' || v_qty);  
END;
```

O detalhe completo (comentários, tratamento de exceção linha a linha) está em database/04\_procedures.sql e database/05\_usage\_examples.sql — este último roda ambas

as functions dentro de SELECTs e ambas as procedures com EXEC, incluindo um caso de exceção tratada (usuário inexistente).

#### **4.3 Fluxo REST → Java → JDBC → Oracle**

PRC\_REGISTER\_CRITICAL\_ALERT é acionada automaticamente pelo backend: ao salvar uma leitura via POST /api/readings, BloodPressureService (Java) chama OracleIntelligenceService, que abre um CallableStatement JDBC e executa a procedure no Oracle. Usuários são casados entre os dois bancos pelo email (os IDs numéricos são gerados de forma independente em cada um).

## 5. Validação prática (Oracle real)

---

Todos os scripts e a integração Java foram testados contra um Oracle Database XE 21c real (container `gvenzl/oracle-xe:21-slim`), não apenas revisados estaticamente. Resultado do teste de ponta a ponta:

- Registro de um novo paciente via API → réplica automática criada em APP\_USER do Oracle (casada por email).
- POST `/api/readings` com 195/128 (CRISE\_HIPERTENSIVA) → linha inserida em BLOOD\_PRESSURE\_READING do Oracle → PRC\_REGISTER\_CRITICAL\_ALERT disparada → alerta CRITICO registrado em SENSOR\_ALERT.
- POST `/api/readings` com 115/75 (NORMAL) → leitura replicada, porém nenhum alerta novo criado (confirma o IF da procedure).
- GET `/api/oracle/users/{id}/summary` e `/delivery-report` retornaram exatamente os dados calculados pelas functions/procedures no banco.

*Evidência completa (saída real do sqlplus) documentada em database/README.md.*

## 6. Entrega do Projeto

---

### 6.1 Código-fonte

Repositório GitHub: <https://github.com/LeoMarsiarelli/Smart-HAS>

Estrutura do monorepo: `/mobile`, `/backend` (inclui integração Oracle), `/web-admin`, `/database` (scripts PL/SQL), `/docs`.

### 6.2 Vídeo de demonstração

Link do vídeo (YouTube, não listado): [INSERIR LINK DO VÍDEO AQUI]

### 6.3 Credenciais de demonstração

| Papel         | Email              | Senha       |
|---------------|--------------------|-------------|
| Administrador | admin@smarthas.com | admin123    |
| Paciente      | maria@smarthas.com | paciente123 |

## 7. Conclusão

---

O Smart HAS consolidou, nesta fase, a interoperabilidade entre mobile, web, API REST e banco Oracle: as mesmas regras de negócio agora vivem tanto em Java quanto em PL/SQL, reutilizáveis por qualquer cliente que converse com o banco. O próximo passo natural do roadmap é levar essa mesma inteligência

para um modelo de Machine Learning treinado com dados históricos reais, mantendo a base sólida de dados e regras construída agora.